ORGANISASI
&
ARSITEKTUR KOMPUTER

NAMA	ITA ALFIRA LALA
NIM	24.11.6127
KELAS	IF04

INFORMATIKA
UNIVERSITAS AMIKOM YOGYAKARTA
2025/2026

1. Analisis Mekanisme Komunikasi I/O Jelaskan dan analisis perbedaan kinerja kompleksitas antara tiga metode komunikasi I/O berikut: Programmed I/O Interrupt-Driven I/O Direct Memory Access (DMA) Dalam analisismu, jelaskan situasi atau skenario sistem di mana masing-masing metode paling efisien digunakan, serta dampaknya terhadap beban CPU.
   Jawab:
   Komunikasi I/O adalah cara CPU dan perangkat I/O bertukar data dan status. Tiga metode utama memiliki perbedaan signifikan dalam kinerja dan kompleksit

   Perbedaan Kinerja dan Kompleksitas

Metode Komunikasi	Mekanisme Dasar	Kompleksitas Controller	Dampak pada Beban CPU
Programmed I/O (PIO)	CPU secara berulang memeriksa status perangkat I/O (polling) hingga perangkat siap. CPU secara langsung mentransfer data per *word*.	Rendah	Sangat Tinggi. CPU terikat (*busy-waiting*), menunggu perangkat I/O.
Interrupt-Driven I/O	Perangkat I/O memberi sinyal interupsi ke CPU saat siap mentransfer data atau setelah operasi selesai. CPU mentransfer data per *word*.	Menengah	Menengah. CPU hanya aktif saat terjadi interupsi, tidak perlu *polling* terus-menerus.
Direct Memory Access (DMA)	Controller DMA mentransfer blok data langsung antara memori dan perangkat I/O tanpa intervensi CPU. CPU hanya terlibat di awal dan akhir transfer.	Tinggi	Sangat Rendah. CPU bebas melakukan tugas lain saat transfer berlangsung.

Situasi Paling Efisien

Metode	Skenario penggunaan Paling Efisien
Programmed I/O (PIO)	Perangkat lambat yang mentransfer data dalam jumlah sangat kecil (misalnya, *keyboard* lama atau *port* serial sederhana). Beban CPU yang tinggi dapat ditoleransi karena *overhead* interupsi lebih buruk daripada *polling* sederhana.
Interrupt-Driven I/O	Perangkat yang relatif lambat atau yang jarang diakses (misalnya, *mouse* atau *printer*). Metode ini efisien ketika *overhead* pemrosesan interupsi lebih rendah daripada waktu yang terbuang oleh *polling* terus-menerus (PIO).
Direct Memory Access (DMA)	Perangkat berkecepatan tinggi (misalnya, SSD, kartu grafis, atau adaptor jaringan). DMA mutlak diperlukan untuk mentransfer blok data besar karena membebaskan CPU, memaksimalkan *throughput* sistem, dan mencegah *bottleneck* I/O.

2. Analisis Peran Controller I/O Modul I/O sering kali dihubungkan dengan device controller yang bertugas mengelola transfer data antara perangkat dan CPU. Analisis bagaimana controller I/O berfungsi sebagai “perantara cerdas” dalam arsitektur sistem komputer, dan jelaskan bagaimana kegagalan atau bottleneck pada controller dapat memengaruhi keseluruhan performa sistem.
   Jawab:
   Controller I/O (*device controller*) bertindak sebagai "perantara cerdas" antara perangkat I/O yang unik dan CPU yang umum. Fungsinya adalah untuk mengisolasi CPU dari detail teknis perangkat keras spesifik.

   Analisis Fungsinya:
   1. Translasi Perintah**:** CPU mengirimkan perintah I/O tingkat tinggi (misalnya, "baca blok 512"). Controller menerjemahkan perintah ini menjadi serangkaian sinyal listrik spesifik yang dipahami oleh perangkat fisik (*drive*).
   2. Manajemen Buffer Lokal**:** Controller sering menyertakan *buffer* data lokal. Ini memungkinkan transfer data antara perangkat dan controller dilakukan dengan kecepatan perangkat, sementara transfer antara controller dan memori/CPU dapat dilakukan pada kecepatan bus. Hal ini mengatasi perbedaan kecepatan yang signifikan.

3. Pengendalian Status dan Kesalahan: Controller melacak status perangkat (misalnya, siap, sibuk, atau gagal) dan melaporkan kesalahan kembali ke CPU (biasanya melalui interupsi). Hal ini membebaskan CPU dari keharusan memantau perangkat secara terus-menerus.
4. Kontrol Alamat dan Waktu: Controller menentukan alamat pada perangkat (misalnya, *track*, *sector* pada HDD) dan mengelola *timing* sinyal I/O yang presisi.

Dampak Kegagalan atau Bottleneck pada Controller

- Kegagalan Total: Jika controller gagal, perangkat yang terhubung dengannya tidak akan dapat berkomunikasi dengan sistem. Ini adalah *single point of failure* (SPOF) untuk perangkat tersebut.
- Bottleneck Performa: Controller bertindak sebagai gerbang *throughput*. Jika bus antara controller dan perangkat lebih cepat daripada bus antara controller dan CPU, atau jika *buffer* internal controller terlalu kecil, controller akan menjadi *bottleneck*.
    - Contoh: Controller SATA lama mungkin hanya mendukung kecepatan transfer 3 Gbps, meskipun SSD yang terhubung mampu mencapai 6 Gbps. Kinerja maksimum sistem akan terbatas pada 3 Gbps.
- Peningkatan Latensi: Jika controller memproses permintaan secara serial atau memiliki manajemen *queue* yang buruk, latensi I/O untuk semua permintaan akan meningkat, memperlambat semua aplikasi yang bergantung pada I/O.

3. Analisis Bus dan Arsitektur I/O Dalam sistem modern, digunakan berbagai jenis bus I/O seperti PCIe, USB, dan SATA. Bandingkan dan analisis arsitektur serta kecepatan transfer data ketiga bus tersebut. Jelaskan pula bagaimana karakteristik fisik dan protokol komunikasi masing-masing memengaruhi efisiensi dan kompatibilitas dengan CPU dan memori utama.
   Jawab:
   Berbagai jenis bus I/O digunakan dalam sistem modern, masing-masing dengan arsitektur dan kecepatan yang dioptimalkan untuk perangkat tertentu.

Perbandingan Arsitektur dan Kecepatan Transfer Data

Bus I/O	Arsitektur	Kecepatan Transfer (contoh)	Protokol Utama
PCI Express (PCIe)	Serial *point-to-point*. Menggunakan jalur independen (*lane*) (x1, x4, x8, x16). Berbasis paket (*packet-based*).	PCIe 5.0 (x16): hingga 64 GB/s	NVMe (untuk *storage*), Protokol kartu grafis, Jaringan kecepatan tinggi
Universal Serial Bus (USB)	Topologi *hub-and-spoke* (Host Controller ke *hub*). Protokol *polling* dan interupsi untuk koneksi *hot-swappable*.	USB 3.2 Gen 2x2: hingga 20 Gbps	Berbasis *Host Controller Interface* (HCI)
Serial ATA (SATA)	Serial *point-to-point* antara Host Controller dan *drive*. Dirancang khusus untuk penyimpanan (*storage*).	SATA III (6 Gbps)	AHCI (untuk HDD/SSD), NCQ

Pengaruh Karakteristik Fisik dan Protokol

1. PCIe (Peripheral Component Interconnect Express):
    - Karakteristik Fisik: Koneksi langsung ke *chipset* atau CPU (terutama untuk jalur x16 kartu grafis), memberikan latensi sangat rendah.
    - Protokol: Protokol NVMe pada PCIe memungkinkan transfer data secara paralel dan sangat efisien, menghilangkan *overhead* protokol SATA/AHCI yang dirancang untuk perangkat mekanis. Dampaknya**:** Sangat efisien untuk SSD berkecepatan tinggi, kartu grafis, dan adaptor jaringan, karena memaksimalkan *throughput* dan meminimalkan latensi.
2. USB (Universal Serial Bus):
    - Karakteristik Fisik: Kabel eksternal, *hot-pluggable*. Sangat fleksibel dan universal.
    - Protokol: Protokol berbasis Host Controller memerlukan *polling* periodik dan *overhead* yang lebih tinggi daripada PCIe. Dampaknya: Ideal untuk kompatibilitas dan perangkat eksternal dengan kecepatan menengah (flash drive, *webcam*, *printer*), namun kurang efisien untuk penyimpanan internal berkecepatan tinggi.

3. SATA (Serial ATA):
    - Karakteristik Fisik: Kabel internal yang dirancang untuk *storage*.
    - Protokol: Protokol AHCI (Advanced Host Controller Interface) memiliki *overhead* yang lebih tinggi dibandingkan NVMe karena dirancang sebagai evolusi dari protokol IDE/ATA untuk HDD. Dampaknya: Efisien dan murah untuk HDD dan SSD SATA tradisional, tetapi menjadi bottleneck kinerja SSD modern, terutama dalam hal IOPS.

4. Analisis Sinkronisasi dan Manajemen I/O Pada sistem multiprogramming, sering terjadi kompetisi antar perangkat I/O dalam mengakses bus atau memori utama. Analisis bagaimana mekanisme sinkronisasi dan scheduling I/O (misalnya menggunakan buffer, queue, atau interrupt priority) dapat menjaga efisiensi sistem serta mencegah terjadinya data inconsistency dan deadlock.
   Jawab:
   Dalam sistem multiprogramming, manajemen I/O yang tepat sangat penting untuk mencegah inkonsistensi data, *deadlock*, dan menjaga efisiensi.

   Mekanisme Sinkronisasi dan Scheduling I/O
   1. Penggunaan Buffer (Buffering)
      - Analisis: *Buffer* adalah area memori sementara yang digunakan untuk menyimpan data saat sedang ditransfer antara perangkat I/O dan memori utama.
      - Fungsi: Menjembatani perbedaan kecepatan dan *burstiness* I/O.
        - Efisiensi Sistem: Memungkinkan CPU dan I/O beroperasi secara paralel. CPU dapat mengisi *buffer* dengan cepat, dan perangkat I/O dapat membacanya dengan kecepatan yang lebih lambat tanpa membuat CPU menunggu (*producer-consumer problem*).
        - Inconsistency: Mengurangi risiko *data inconsistency* karena data berada dalam area yang terkendali saat proses transfer.
   2. Penggunaan Queue (Scheduling I/O)
      - Analisis: Permintaan I/O dari berbagai proses dimasukkan ke dalam *queue* perangkat sebelum dieksekusi. Algoritma *disk scheduling* (misalnya, FCFS, SSTF, SCAN) digunakan untuk mengatur urutan eksekusi.
      - Fungsi: Optimasi gerakan *head* disk (untuk HDD) atau meningkatkan efisiensi *flash memory* (untuk SSD).
        - Efisiensi Sistem: Algoritma seperti SCAN meminimalkan waktu *seek* pada HDD, yang secara drastis meningkatkan *throughput* sistem. Pada SSD, *scheduling* membantu pengelompokan operasi tulis untuk efisiensi *flash memory*.
        - Deadlock: Penggunaan *queue* dan *resource management* yang ketat (misalnya, membatasi jumlah *outstanding requests*) membantu

mencegah *deadlock* I/O, di mana dua atau lebih proses saling menunggu sumber daya I/O yang sudah dipegang oleh yang lain.

3. Prioritas Interupsi (*Interrupt Priority*)
    - Analisis: Interupsi diberi tingkat prioritas. Interupsi I/O berkecepatan tinggi atau *real-time* (seperti disk atau jaringan) diberi prioritas lebih tinggi daripada interupsi dari perangkat lambat (seperti *mouse*).
    - Fungsi: Memastikan *event* I/O yang mendesak segera ditangani.
        - Efisiensi Sistem: Mempercepat respons sistem terhadap I/O kritis, sangat penting untuk sistem *real-time*.
        - Inconsistency/Deadlock: Memastikan sumber daya yang sangat penting (misalnya bus I/O atau blok memori) dibebaskan dan dialokasikan ulang dengan cepat, mengurangi risiko penundaan yang dapat menyebabkan kondisi *race* atau *deadlock* sementara.

5. Analisis Desain Sistem I/O Terpadu Seorang perancang sistem ingin mengembangkan komputer embedded untuk sistem otomasi industri yang memerlukan banyak sensor dan aktuator real-time. Analisis bagaimana Anda akan merancang arsitektur I/O yang efisien untuk sistem ini. Sertakan pertimbangan terhadap: Jenis metode transfer data Penggunaan interrupt dan DMA Kebutuhan real-time dan reliabilitas Skalabilitas jumlah perangkat I/O

   Jawab:

   Sistem harus didasarkan pada CPU dengan dukungan DMA yang kuat, bus PCIe untuk perangkat I/O utama (seperti *network interface*), dan RTOS yang mendukung *preemptive scheduling* dan *interrupt priority* untuk memastikan transfer data *deterministic* dan *real-time*.

   Rancangan Arsitektur I/O yang Efisien

   Pertimbangan 1: Jenis Metode Transfer Data
   - Pilihan: Direct Memory Access (DMA) adalah pilihan utama.
   - Justifikasi: Sensor dan aktuator akan menghasilkan aliran data yang berkelanjutan dan harus diproses dalam waktu yang ketat. DMA harus digunakan untuk mentransfer blok data sensor dari perangkat ke memori utama (RAM) dan sebaliknya untuk perintah aktuator. Ini membebaskan CPU untuk fokus pada *logic* kontrol *real-time* daripada manajemen data.

Pertimbangan 2: Penggunaan Interrupt dan DMA

- Interrupt: Digunakan secara minimal dan hanya untuk *event* kritis atau notifikasi penyelesaian tugas (*task completion*). Contoh: interupsi kritis saat batas suhu terlampaui.
- DMA: Digunakan untuk semua transfer data rutin bervolume tinggi. Controller DMA harus diprioritaskan tinggi, dan perlu memiliki *channel* DMA khusus untuk perangkat *real-time* yang paling penting.

Pertimbangan 3: Kebutuhan Real-Time dan Reliabilitas

1. Bus Deterministik: Pilih bus I/O dengan waktu akses yang dapat diprediksi, seperti PCI Express (untuk *throughput* tinggi) atau bus industri yang khusus (*Fieldbus/EtherCAT*) yang menawarkan *deterministic latency*.
2. Reliabilitas: Gunakan Controller I/O yang *redundant* jika memungkinkan, dan sertakan mekanisme *Error Correction Code* (ECC) pada memori yang digunakan untuk *buffer* I/O untuk memastikan integritas data sensor dan perintah aktuator.
3. Prioritas I/O: Implementasikan Interrupt Priority Controller (PIC) yang ketat dan sistem *preemptive scheduling* di OS *real-time* (RTOS) untuk memastikan *event* I/O kritis selalu didahulukan.

Pertimbangan 4: Skalabilitas Jumlah Perangkat I/O

1. Arsitektur Modular: Gunakan arsitektur bus seperti PCIe yang dapat diperluas secara modular (misalnya melalui *riser card* atau *backplane*) untuk menampung lebih banyak *device controller*.
2. Bus Cabang (Branching Bus): Menggunakan *Fieldbus* atau jaringan *Industrial Ethernet* memungkinkan ratusan sensor dan aktuator dihubungkan secara *daisy-chain* atau *ring* ke satu *gateway controller* utama. Ini mengurangi jumlah koneksi langsung ke CPU tetapi menggeser kompleksitas ke *protocol management*.